Offline AI Proctoring System

A privacy-focused, offline-first examination monitoring platform designed to detect and analyze potentially suspicious activities during computer-based exams. The system operates entirely on the candidate's local machine using a **Streamlit** dashboard, ensuring data privacy and eliminating the need for continuous internet connectivity.

This project integrates **Multimodal Vision LLMs**, **Multi-Agent Systems (CrewAI)**, and **Retrieval-Augmented Generation (RAG)** to provide a comprehensive, human-in-the-loop proctoring solution.

📌 Project Scope & Core Principles

1. Privacy-First & Offline-First

All video processing, object/anomaly detection, embedding, vector search, and report generation happen **locally** on the host machine using Ollama and local libraries. No exam data or video feeds are sent to external cloud servers.

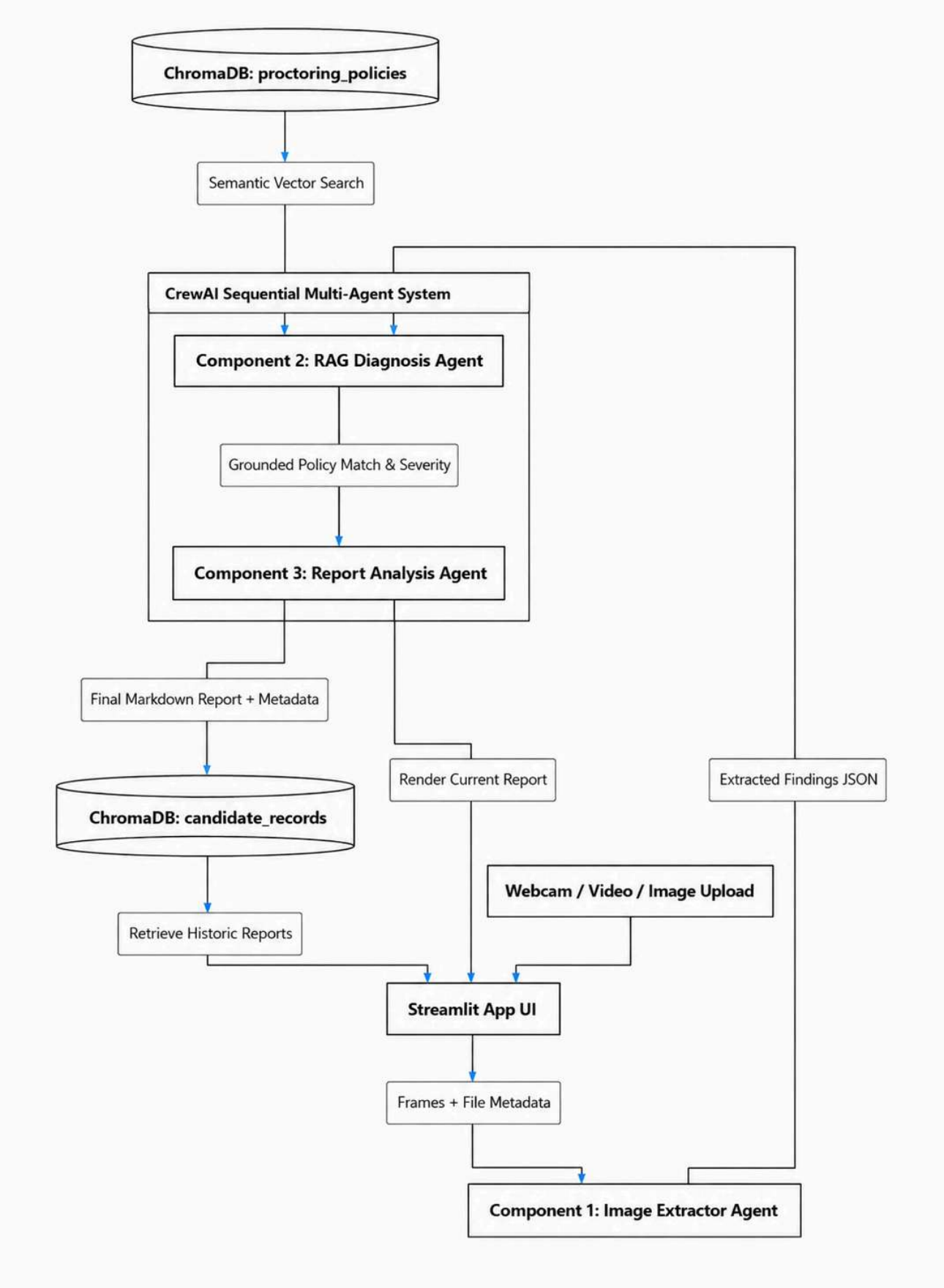
2. High-Accuracy Multimodal Detection

Instead of relying solely on heuristic bounding-box models, this system leverages a local **Multimodal Vision LLM** (e.g., llava via Ollama) to inspect keyframes. This allows the system to accurately detect context, verify timestamps/file metadata, identify prohibited objects (like cell phones), and evaluate the candidate's environment.

3. Human-in-the-Loop Decision Making

AI-generated flags are **not** treated as definitive proof of cheating. The system acts as an auditor that flags anomalies and compiles them into a structured report. The final decision remains with a human reviewer via the Streamlit dashboard.

System Architecture



Flow of Operations:

1. **Input Submission**: The user uploads candidate exam images/videos or processes a webcam stream in the **Streamlit** app.
2. **Sequential Multi-Agent Pipeline (CrewAI)**:
    - **Image Extractor Agent**: Inspects the uploaded image/video frames, extracts file metadata (date, time, format), and uses a local Multimodal LLM to detect objects (phones, additional people) or pose anomalies.
    - **RAG Diagnosis Agent**: Chunk-embeds the Extractor Agent's findings and performs a semantic search against the proctoring rules store in ChromaDB. It maps detected events to institutional policies (e.g., gaze deviation severity).
    - **Report Analysis Agent**: Compiles the raw event timeline and the retrieved policy rules into a clean, human-readable markdown report with recommended actions.
3. **Archival & Visualization**: The Streamlit interface displays the generated report and saves the candidate information, session metadata, and final report into a unified database in ChromaDB for future audit reviews.

🤖 CrewAI Multi-Agent Design

The system runs a strict **sequential process** (where the output of each agent is fed as context into the next):

1. Image Information Extractor Agent

- **Role**: Multimodal Image & Metadata Analyzer
- **Goal**: Analyze raw image/video frames and file metadata to extract details (timestamps, dates) and detect visual anomalies (faces, gaze direction, cell phones, or notes).
- **Backstory**: An expert in computer vision and multimodal image analysis. This agent inspects files and utilizes a local Vision LLM to return a structured JSON log of events.
- **Tools**: Local Metadata Parser, Ollama Multimodal Vision Tool.

2. RAG Diagnosis Agent

- **Role**: Proctoring Policy Matcher & Compliance Auditor
- **Goal**: Perform semantic search on extracted image info to map anomalies to institutional proctoring guidelines.
- **Backstory**: A detail-oriented compliance officer. It takes the structured JSON

findings from the Extractor Agent, queries the policies collection in ChromaDB, and aligns events with specific violation classes and severity levels.

- **Tools**: ChromaDB Policy Search Tool.

3. Report Analysis Agent

- **Role**: Post-Exam Summary Writer
- **Goal**: Synthesize the raw event data and RAG policy context into a professional, human-readable review report.
- **Backstory**: A technical writer specializing in educational integrity. This agent translates complex AI logs into a narrative summary, including a chronological timeline of incidents, severity indexes, and recommended actions for the human reviewer.
- **Tools**: Report Builder.

🛠️ Technology Stack & Frameworks

Category	Technology / Framework	Purpose
User Interface	Streamlit	Interactive local web dashboard for running audits, managing uploads, and viewing report histories.
Agent Orchestration	CrewAI	Managing sequential agent execution, task delegation, and communication.
Local LLM & Vision	Ollama	Running local LLMs: - llava (multimodal vision for image extraction) - llama3.1 (text processing & report writing)
Vector Database	ChromaDB	1) **RAG Knowledge Base**: Storing academic policies and incident definitions. 2) **Candidate Vault**: Storing candidate details, metadata, and final audit reports.
Embeddings	Sentence-Transformers (all-MiniLM-L6-v2)	Creating local semantic vectors of policies and event logs.
Computer Vision Utilities	OpenCV	Frame extraction from video uploads and camera management.

📂 Database Schema & Unified Store

To keep the local stack lightweight and consolidated, we use **ChromaDB** as the primary storage engine using two collections:

1. proctoring_policies (Knowledge Base Collection)

Stores chunks of the institution's exam guidelines and event definitions.

- **Document Content**: "A phone detection event is classified as High Severity. Immediate flag and reviewer escalation required."
- **Metadata**: { "policy_id": "rule_03", "event_type": "phone_detected", "severity": "high" }

2. candidate_records (Candidate Archive Collection)

Stores the mapping of candidate info, session metadata, and final reports.

- **Document Content**: *Full markdown text of the post-exam report.*
- **Metadata**:

```
{
   "candidate_id": "cand_987",
   "candidate_name": "John Doe",
   "session_id": "exam_123",
   "exam_date": "2026-08-13",
   "risk_level": "Medium",
   "extracted_events": "phone_detected: 1, gaze_deviation: 3",
   "reviewer_status": "Pending Action"
}
```

Using ChromaDB metadata allows filtering directly by candidate_id or risk_level to display historical reports in the Streamlit UI.

🚀 Stepwise Implementation Roadmap

Phase 1: Environment & Ollama Setup

1. Initialize the Python environment with required packages (streamlit, crewai, chromadb, opencv-python, sentence-transformers).
2. Install Ollama and pull the models:

```
ollama pull llava:7b
ollama pull llama3.1:8b
```

Phase 2: Knowledge Base RAG Setup

1. Write 5–10 proctoring policy markdown files defining rule violations, severity mappings, and incident styles.
2. Build a python ingestion script to read these files, chunk them, embed them using all-MiniLM-L6-v2, and populate the ChromaDB proctoring_policies collection.

3. Test semantic querying against this database.

Phase 3: CrewAI Sequential Assembly

1. Define the 3 Agents (Image Extractor, RAG Diagnosis, Report Analyst).
2. Write tasks with strict sequential schemas:
    - **Task 1 (Extractor)**: Parse image metadata, feed image/frames to Ollama llava, and extract detected anomalies as structured JSON output.
    - **Task 2 (RAG)**: Take the JSON output, run semantic search against ChromaDB proctoring_policies collection, and match rules.
    - **Task 3 (Analyst)**: Synthesize the matched policies and event timelines into a structured audit report.

Phase 4: Streamlit UI Development

1. Build an upload dashboard allowing users to select an image/video or capture via webcam.
2. Integrate the sequential CrewAI run, showing real-time logs of agent tasks.
3. Build the "Candidate Vault" page, which queries ChromaDB's candidate_records collection to display historical candidate details and generated reports.
4. Implement a simple "Approve / Flag" interface that writes reviewer feedback back to the candidate's metadata in ChromaDB.

Phase 5: Testing & Fine-Tuning

1. Test with demo images (normal behaviour, phone in hand, looking away).
2. Adjust Vision LLM prompting parameters to prevent hallucinations and improve metadata extraction accuracy.
3. Benchmark latency for keyframe processing and optimize the frame-sampling rate.